Artifacts and Evidence

Artifacts and Evidence I

This section reports what the project has actually produced and how each quantitative claim is grounded. Every number below is substituted at render time from the deterministic producer, which reads the repository state at commit 56cc123db, so each figure here is regenerated from the artifacts it describes rather than transcribed.

Model-Family Coverage I

GNN ships a curated corpus of model families that exercise the language across the difficulty gradient from minimal parser fixtures to full multi-agent and scaling studies. The manifest registers 9 families (basics, discrete, continuous, hierarchical, multiagent, precision, structured, gridworld, scaling-study) across 9 target directories, and `input/gnn_files` holds 12 corpus directories containing 38 concrete example models. The three sets are not coextensive, and the manuscript does not treat them as one. Registered in no family: `input/gnn_files/learning`, `input/gnn_files/recursive`, `input/gnn_files/thrml` (3 of the 12 corpus directories), covered in sec. 7. Registered but outside the scanned tree: none, because all 9 target directories are themselves `input/gnn_files` corpus directories. Each family declares the simulation frameworks it is meant to drive, which is what lets the same

text model fan out across the executable-model leg of the Triple Play (Smékal and Friedman 2023). The families and their declared frameworks are enumerated below.

The exemplar corpus partitions cleanly by model kind, and the partition is itself a token: 29 of the 38 example specifications are discrete-state — the discrete categorical kind and its composed variants — and 9 are continuous linear-Gaussian specifications in the continuous family folder. The cold-start index `input/gnn_files/INDEX.md` catalogues every exemplar with its kind, its task folder, and a suggested reading order, and is the fastest surface for a reader who wants to move from this prose to a runnable file of a given kind.

Model-Family Coverage I

Families that declare a capability axis (the continuous family declares the continuous-render axis) carry a backend split in tbl. 1 generated directly from the registry's declared capability flags. This avoids a manually copied registry table; actual model admission still depends on the full parameter and composition checks recorded by the renderer. The generated flags describe declared capability, while current-run receipts record which selected backends were actually attempted, unsupported, skipped, or failed. The profiled cross-framework reference uses the continuous family, so it does not establish categorical coverage across the registry (tbl. 4 is the per-kind key).

Model-Family Coverage I

Table 1: Model families declared in `input/model_family_manifest.json` with the simulation frameworks each family targets and a generated description. The *Frameworks* column is the family's declared targets, verbatim from the manifest; the capability splits appended to some descriptions are generated from `src/gnn/render/framework_registry.py` flags, not authored in the manifest, so they cannot disagree with the registry. Read with tbl. 3 (what each backend is) and fig. 6 (the same coverage as a matrix).

Family	Frameworks	Description
`basics`	pymdp	Minimal perception fixtures used for parser and validator smoke coverage.
`discrete`	pymdp, jax	Discrete POMDP, passive HMM filtering and episodic contingent T-maze fixtures.
`continuous`	jax, numpyro, stan, rxinfer, ngclearn	Continuous-state linear-Gaussian fixtures (F/H/Q/R + Gaussian prior; continuous_navigation closes the loop on beliefs; ngc-learn LGSSM exemplar for the ngclearn backend). Native on RxInfer.jl, JAX, PyTorch, NumPyro, Stan, cpomdp, ngc-learn; reported unsupported (not failed) on PyMDP, ActiveInference.jl, DisCoPy, bnlearn, THRML.
`hierarchical`	jax	Declared block-reset hierarchy and approximate 10/100-clock soft-message controller fixtures.
`multiagent`	rxinfer	Multi-agent coordination and swarm fixtures.
`precision`	pymdp	Precision weighting and curiosity-driven fixtures.
`structured`	pymdp	Structured factor graph and posterior fixtures.
`gridworld`	pymdp, rxinfer, activeinference_jl	Gridworld POMDP fixture used for cross-framework acceptance checks.

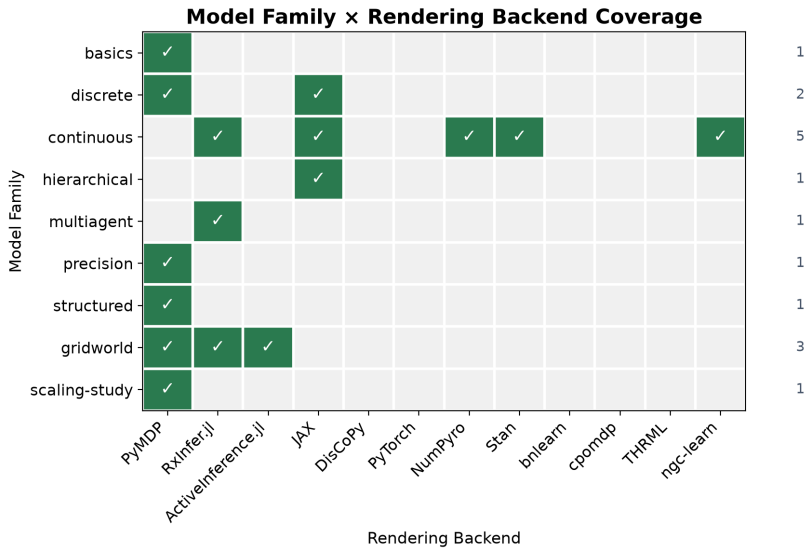
Model-Family Coverage I

Family	Frameworks	Description
`scaling-study`	pymdp	PyMDP scaling-study fixtures, sampled conservatively for acceptance.

Model-Family Coverage I

The family-by-framework structure is shown in fig. 6, which renders the coverage matrix directly from the family registry rather than from a hand-maintained table.

Model-Family Coverage I



Declared coverage cells: 16 of 108 (9 families × 12 backends); the right-hand count is how many backends each family declares

Figure 6: Model-family coverage across the registered rendering backends: one row per family from `input/model_family_manifest.json`, one column per backend from `src/gnn/render/framework_registry.py`, and a green cell wherever the family declares that backend in its `frameworks` field. The right-hand count states how many backends each family declares; the grid is deliberately sparse — most families declare a single

backend, and only continuous, discrete, and gridworld declare several. Read it as declared intent, not as profiled outcomes: the gates described below supply the outcomes. The matrix is generated from the two registries at commit 56cc123db.

Model-Family Coverage I

These families are not illustrative prose: they are the inputs over which the parser, the type checker, and the cross-framework code generators are exercised, and they are the substrate for the reliability gates described next.

Semantic-Fidelity and Cross-Framework Reliability Gates I

Two command-defined gates check distinct parts of GNN's shared-source contract on selected family fixtures. Both live under `scripts/` and use the family manifest described above. Their actual source-bound command receipts determine the scope and outcome of verification; the presence of a gate is not itself a passing result.

The semantic-fidelity gate, `scripts/run_semantic_fidelity_gate.py`, checks parse–serialize–parse preservation using the versioned `gnn_semantic_contract_v2` contract. It compares model identity, variables, edges, parameter names, shapes, values and metadata, equations, timing, and ontology mappings; the default strict round trip is JSON. Parameter values and their axis order contribute to the contract hash, so changing values without changing shape changes the hash. The digest binds the represented contract; it does not prove that extraction was correct, that generated inference implements the intended

mathematics (Da Costa et al. 2020), or that independent numerical executions agree. The gate reports scoped fidelity per selected model and format.

The cross-framework reliability gate, `scripts/run_cross_framework_reliability.py`, takes a single GNN model and renders it across multiple simulation backends, then checks that the resulting executable models agree on the structure they were generated from. The reference comparison runs on the continuous family across JAX, NumPyro, RxInfer.jl — 3 independent Active Inference engines spanning the Python and Julia ecosystems (Heins et al. 2022). The family declares 5 backends (JAX, NumPyro, Stan, RxInfer.jl, ngc-learn); Stan, ngc-learn is declared but not among the 7 frameworks the gate profiles, so it is excluded from the comparison. Because the same source model drives all 3 renderings, source identity helps investigate disagreement. The gate compares shared matrix shapes, trace

lengths, timesteps, and reported seeds, recording absent comparable fields separately. These checks do not establish numerical equivalence: a mismatch may arise from extraction, code generation, runtime or toolchain behavior, inference mode, dtype, or a backend defect. Numerical comparisons require compatible models and inference semantics, explicit tolerances, and source-bound numerical witnesses.

Both gates are stated here as commands you can run, not as asserted pass counts. The manuscript deliberately does not quote a fixed number of passing checks: an authoritative result is a retained command receipt bound to its source revision, selected corpus, environment, and artifacts, and binding a frozen count into prose would invite exactly the drift the auto-injection contract exists to prevent.

A third interchange check extends the same discipline across repositories: `scripts/run_geo_interchange_checks.py` validates the committed pin in `.github/gnn-pair.json` against a selected GEO-INFER checkout and replays exported GNN artifacts — the tracked gridworld, a compiled H3 stay/diffuse model, a rectangular Gaussian, and an explicit factored fixture — inside the GEO environment, writing its receipts even on failure. Like the gates above it is stated as a command, not as a pass count, and its pinned pair (`ActiveInferenceInstitute/GEO_INFER` at a recorded revision) is what makes "interchange" a checkable claim rather than a promise.

Repository Scale I

The repository's scale is itself evidence of the surface that the gates and pipeline cover, and it is reported in fig. 7 directly from the tracked files at commit 56cc123db.

Repository Scale I

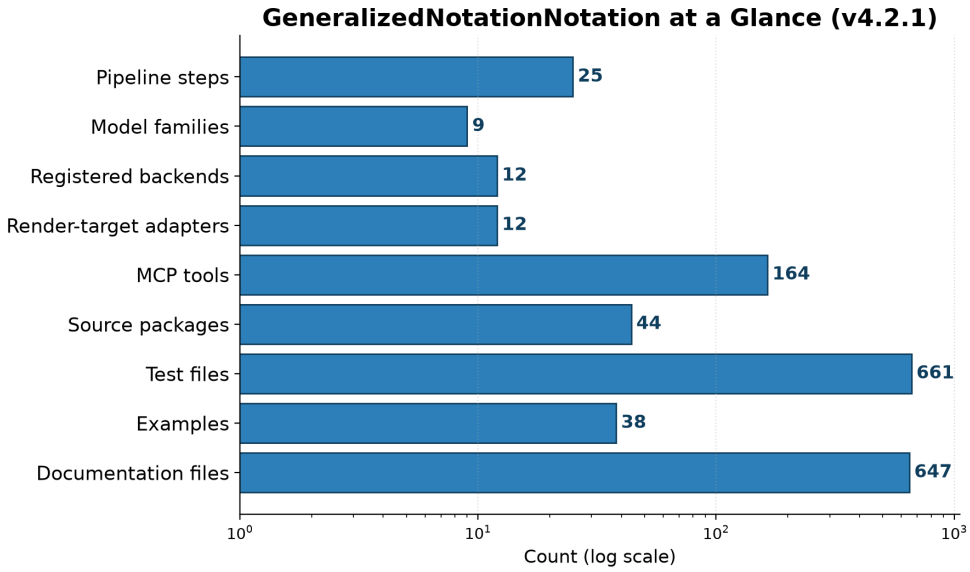


Figure 7: Repository-scale counts on a logarithmic axis: pipeline steps, model families, registered backends, render-target execution adapters, Model Context Protocol tools, source packages, test files, example models, and documentation files. Every bar is annotated with its exact value, and every value is a producer token read from `output/data/manuscript_variables.json` at commit 56cc123db — the same token

map that substitutes the prose counts. The figure-freshness suite binds the committed image digest and its recorded producer inputs and counts; semantic content and visual correctness require their own review. The two backend bars are deliberately distinct: *registered backends* (12) counts render targets, *render-target adapters* (12) counts the render-target subset with implemented Step 12 execution

adapters. It excludes the separate execution-only Lean bridge and is not a count of observed native runs. Read the chart as the scale of the surface the pipeline maintains, not as a quality measure.

Repository Scale I

The test suite comprises 661 test files containing 6817 test functions, exercising a source base of 767 Python files across 44 packages (226030 lines of source). The pipeline's step modules — the thin orchestrators named in tbl. 2 — number 25, one per numbered step. The Model Context Protocol surface — which exposes GNN's capabilities to external agents and tools — provides 164 tools across 36 modules. The pipeline itself runs as 25 steps (0–24), and 7 figure artifacts from the rendering of figures, models, and reports are committed under `output/`, of which 7 are the manuscript's own.

Claim Discipline I

A claim is manuscript-ready only when it is bound to a verifiable artifact. Concretely, every claim in this manuscript must rest on one of four support types:

Claim Discipline I

- A retained passing test or validator command receipt, bound to the source revision, selected inputs, environment, and artifacts — for example a scoped execution of the semantic-fidelity or cross-framework gates above.
- A generated output produced by a deterministic producer, such as the figures rendered from the family and repository scans, or the token values emitted by the manuscript-variable producer that backs every number on this page.
- A source ledger, manifest, or configuration file that fixes a declared value or capability; actual execution still requires its own result receipt.
- A resolved entry in `references.bib` for any external-literature claim (Friston 2010; Parr et al. 2022).

Claim Discipline I

The pipeline records its own evidence trail under `output/`. The run-level summary is written to `output/PIPELINE_REPORT.md`, and the per-step execution record — including which steps ran, their status, and their artifacts — is captured in `output/00_pipeline_summary`. Readers should use those current-run records and their identity bindings to assess actual pipeline outcomes. Repository counts and declared capability tokens describe the source surface and do not by themselves establish native execution or scientific correctness. When a value would otherwise need a literal number with no producer behind it, the discipline is to omit the number rather than to hard-code it.